

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática

INF761 — DESARROLLO DE VIDEOJUEGOS

EXAMEN DE TERCER PARCIAL

Examen práctico · Desarrollo de un juego de plataformas 2D en Unity 6

Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Gestión	I / 2026
Modalidad	Práctico individual — desarrollo + defensa en video
Motor / entorno	Unity 6 (6000.x LTS) · proyecto 2D
Período de desarrollo	7 días calendario desde la publicación del enunciado
Valoración total	100 puntos

1. Competencia evaluada

El presente examen evalúa la capacidad del estudiante para integrar, en un único producto jugable y publicable, los componentes desarrollados a lo largo del semestre: control de personaje, diseño de niveles con Tilemap, cámara con Cinemachine, mecánicas mediante ScriptableObjects, sistemas de juego (GameManager y estados), interfaz de usuario, audio, pulido (game juice) y empaquetado multiplataforma con Build Profiles.

RESULTADO ESPERADO

El estudiante construye, prueba y defiende un juego de plataformas 2D completo y ejecutable, demostrando dominio del flujo de producción en Unity 6 y capacidad de explicar y justificar sus decisiones técnicas.

2. Descripción del proyecto

Cada estudiante desarrollará un juego de plataformas 2D de desplazamiento lateral (side-scroller) con al menos un nivel jugable de principio a fin. El objetivo del jugador es atravesar el nivel recolectando objetos, evitando enemigos y peligros, y alcanzando la meta de salida.

Tema sugerido — “Reliquias del Cerro”

Un personaje explora las galerías de una mina abandonada en busca de reliquias de cristal. Debe saltar entre vagonetas y andamios, esquivar derrumbes y criaturas de las cavernas, y llegar a la salida con la mayor cantidad de cristales posible.

LIBERTAD CREATIVA

El tema es solo una sugerencia. El estudiante puede reescenificar libremente el juego (personajes, arte, ambientación) siempre que cumpla todos los requisitos funcionales del punto 4. La calificación evalúa funcionalidad y arquitectura, no la calidad artística de los assets.

3. Requisitos técnicos obligatorios

- **Motor:** Unity 6 (6000.x). El proyecto debe abrir sin errores en esta versión.
- **Entrada:** Input System (paquete nuevo) configurado con un Input Action Asset, o como mínimo Project Settings → Active Input Handling compatible.
- **Cámara:** Cinemachine para el seguimiento del jugador.
- **Escenario:** Tilemap (Grid + Tilemap Renderer + Tilemap Collider 2D) para construir el nivel.
- **Datos:** Al menos un ScriptableObject para parametrizar el juego (p. ej. configuración del jugador, definición de enemigo o de coleccionable).
- **UI:** TextMeshPro para todos los textos de interfaz (HUD y menús).
- **Empaquetado:** Build Profiles de Unity 6 para generar un ejecutable de escritorio (Windows o WebGL).
- **Control de versiones:** Repositorio Git en GitHub con .gitignore de Unity y commits incrementales (no se admite un único commit final).

4. Requisitos funcionales

El juego debe implementar los siguientes bloques. Cada bloque está vinculado a la rúbrica del punto 7.

Bloque A — Control y movimiento del personaje

1. Movimiento horizontal con aceleración/desaceleración (no instantáneo).
2. Salto basado en física (Rigidbody2D) con detección de suelo (ground check).
3. Al menos una mejora de salto: **salto variable** (altura según duración de la pulsación), **coyote time** o **doble salto**.
4. El personaje voltea su orientación (flip) según la dirección de movimiento.

Bloque B — Diseño del nivel

1. Un nivel completo construido con Tilemap, con inicio y meta claramente definidos.
2. Variedad de plataformas (suelo, plataformas elevadas y al menos un desafío de salto exigente).
3. Límites de mundo que impiden que el jugador salga del área jugable.

Bloque C — Cámara

1. Cámara Cinemachine que sigue al jugador con suavizado (damping).
2. Confinamiento de la cámara a los límites del nivel (*Cinemachine Confiner*) para no mostrar el vacío fuera del escenario.

Bloque D — Mecánicas y contenido

1. Coleccionables (p. ej. cristales/monedas) que incrementan el puntaje al recogerse.
2. Al menos **dos tipos de amenaza**: un enemigo con comportamiento (patrulla o persecución básica) y un peligro estático/ambiental (pinchos, caída a vacío, lava, etc.).
3. El contacto con enemigos o peligros aplica daño o causa la muerte del jugador.
4. Al menos un checkpoint o sistema de respawn en el nivel.
5. Una meta/salida que dispara la condición de victoria del nivel.

Bloque E — Sistemas y estados de juego

1. Sistema de vidas o de salud del jugador, con muerte y reaparición.
2. Un **GameManager** que controle los estados del juego: jugando, game over y victoria.
3. Reinicio del nivel tras game over y transición a pantalla de victoria al completar el nivel.

Bloque F — Interfaz de usuario (UI)

1. HUD en partida que muestre, como mínimo, puntaje y vidas/salud.
2. Menú principal con botón de Jugar y Salir.
3. Menú de pausa (tecla Esc) con opciones de Reanudar y Volver al menú.
4. Pantallas de Game Over y Victoria con opción de reintentar o volver al menú.

Bloque G — Audio, animación y pulido

1. Música de fondo en bucle durante la partida.
2. Efectos de sonido para al menos: salto, recolección y daño/muerte.
3. Animator del personaje con, como mínimo, estados de idle, correr y salto.
4. Al menos un elemento de **game juice**: partículas, screen shake, efecto de recolección o feedback visual de daño.

5. Estructura de scripts sugerida

No es obligatorio respetar estos nombres exactos, pero se valora una arquitectura clara con responsabilidades separadas. Una organización razonable de componentes es:

```
Scripts/
├── Player/
│   ├── PlayerController2D.cs // movimiento, salto, ground check, flip
│   └── PlayerHealth.cs      // vidas/salud, daño, muerte, respawn
├── Gameplay/
│   ├── Collectible.cs      // suma puntaje al recolectarse
│   ├── EnemyPatrol.cs      // patrulla / daño por contacto
│   ├── Hazard.cs           // peligro estático (pinchos, vacío...)
│   ├── Checkpoint.cs       // punto de reaparición
│   └── LevelExit.cs         // meta -> condición de victoria
├── Systems/
│   ├── GameManager.cs      // estados: Playing / GameOver / Win
│   ├── UIManager.cs        // HUD, menús, pausa
│   └── AudioManager.cs     // música y SFX
└── Data/
    └── PlayerConfigSO.cs    // ScriptableObject de parámetros
```

BUENAS PRÁCTICAS EVALUADAS

Sin errores ni warnings críticos en consola · uso de [SerializeField] en lugar de campos públicos sueltos · separación entre lógica de juego y presentación · ausencia de código muerto o comentado en exceso.

6. Entregables

La entrega se realiza en el repositorio de GitHub del estudiante e incluye:

1. **Repositorio Git** público o con acceso al docente, con el proyecto Unity completo (sin la carpeta Library, excluida por .gitignore) y commits incrementales fechados dentro del período de desarrollo.
2. **Ejecutable (build)** funcional generado con Build Profiles, comprimido en .zip dentro de una carpeta /Build o publicado como Release de GitHub.
3. **Video de gameplay** de 2 a 4 minutos que muestre el juego completo (enlace en el README).
4. **README.md** con: nombre del juego, controles, instrucciones de ejecución, lista de requisitos cumplidos y enlace al video.
5. **Defensa oral** donde el estudiante ejecuta su build.

7. Rúbrica de evaluación (100 puntos)

Bloque	Criterio	Puntos
A	Control y movimiento: caminar con aceleración, salto físico con ground check y al menos una mejora (salto variable / coyote time / doble salto), flip.	18
B	Diseño del nivel con Tilemap: nivel completo, variedad de plataformas y desafío de salto, límites de mundo.	10
C	Cámara Cinemachine con seguimiento suavizado y confinamiento a los límites del nivel.	6
D	Coleccionables con puntaje, dos tipos de amenaza (enemigo + peligro), daño/muerte, checkpoint y meta de victoria.	22
E	Sistemas y estados: vidas/salud, GameManager (Playing/GameOver/Win), reinicio y transición de victoria.	12
F	UI con TextMeshPro: HUD, menú principal, menú de pausa y pantallas de Game Over / Victoria.	12
G	Audio (música + SFX), Animator del personaje (idle/correr/saltar) y al menos un efecto de game juice.	10
—	Empaquetado y entrega: build ejecutable con Build Profiles, repositorio con commits incrementales, README y video.	10
TOTAL		100

¡Éxitos! — M. Sc. Huáscar Fedor Gonzales Guzmán · INF761 Desarrollo de Videojuegos